Regular Expressions (Regex) Cheat Sheet

A complete reference of symbols, quantifiers, character classes, anchors, groups, and flags used in regular expressions.

---

1. What Is a Regex?

A regular expression (regex) is a sequence of characters that defines a search pattern, used for matching, searching, and replacing text. It is NOT related to statistical regression.

2. Basic Metacharacters

Symbol	Name	Meaning	Example	Matches
`.`	Dot	Any single character (except newline by default)	c.t	cat, cot, c9t
`\`	Backslash	Escapes a special character, or starts a shorthand class	\.	a literal dot '.'
`|`	Pipe / Alternation	OR — matches left or right side	cat|dog	cat, dog
`( )`	Group	Groups a subpattern; also captures it	(ab)+	ab, abab
`[ ]`	Character class	Matches ONE character from the set	[abc]	a, b, or c

3. Character Classes & Shorthand

Symbol	Meaning	Example	Matches
`[abc]`	a, b, or c	[abc]+	a, bc, cab
`[^abc]`	NOT a, b, or c	[^abc]	d, x, 9
`[a-z]`	Any lowercase letter (range)	[a-z]+	hello
`[A-Z]`	Any uppercase letter	[A-Z]	H
`[0-9]`	Any digit (range form)	[0-9]{3}	123
`\d`	Any digit — same as [0-9]	\d\d	42
`\D`	Any NON-digit	\D+	abc, !!
`\w`	Word character: letters, digits, underscore	\w+	hello_123
`\W`	Any NON-word character	\W	@, space, !
`\s`	Whitespace: space, tab, newline	a\sb	a b
`\S`	Any NON-whitespace character	\S+	hello

4. Quantifiers — “How Many Times”

Symbol	Meaning	Example	Matches
`*`	0 or more times	ab*	a, ab, abbb
`+`	1 or more times	ab+	ab, abbb (not 'a')

Symbol	Meaning	Example	Matches
`?`	0 or 1 time (optional)	colou?r	color, colour
`{n}`	Exactly n times	\d{3}	123
`{n,}`	n or more times	\d{2,}	12, 123456
`{n,m}`	Between n and m times	\d{2,4}	12, 123, 1234
`*?  +?  ??`	Lazy (non-greedy) versions — match as little as possible	<.+?>	matches shortest tag, not whole string

5. Anchors & Boundaries

Symbol	Meaning	Example	Matches
`^`	Start of string (or line, in multiline mode)	^Hello	'Hello' only at start
`$`	End of string (or line, in multiline mode)	world$	'world' only at end
`\b`	Word boundary	\bcat\b	'cat' as a whole word, not 'category'
`\B`	NOT a word boundary	\Bcat\B	'cat' inside a larger word

6. Groups, Capturing & Alternation

Symbol	Meaning	Example
`( )`	Capturing group — remembers the match for later use	(\d{3})-(\d{4})
`(?: )`	Non-capturing group — groups without saving the match	(?:ab)+
`(?<name> )`	Named capturing group	(?<year>\d{4})
`\1, \2 ...`	Backreference to a previously captured group	(\w+)\s\1 matches repeated words
`a|b`	Alternation — match a OR b	(cat|dog|bird)

7. Lookaround (Advanced Assertions)

Symbol	Meaning	Example
`(?=...)`	Positive lookahead — must be followed by ...	foo(?=bar) matches 'foo' only if 'bar' follows
`(?!...)`	Negative lookahead — must NOT be followed by ...	foo(?!bar) matches 'foo' only if 'bar' doesn't follow
`(?<=...)`	Positive lookbehind — must be preceded by ...	(?<=\$)\d+ matches digits after a '$'
`(?<!...)`	Negative lookbehind — must NOT be preceded by ...	(?<!\$)\d+ matches digits not after a '$'

8. Common Flags / Modifiers

Flags change how the whole pattern behaves. Syntax varies slightly by language (shown below for Python's `re` module).

Flag	Python Syntax	Meaning
Case-insensitive	re.IGNORECASE / re.I	'cat' matches 'Cat', 'CAT', 'cAt'
Multiline	re.MULTILINE / re.M	^ and $ match start/end of EACH line, not just the whole string
Dot-all	re.DOTALL / re.S	'.' also matches newline characters
Verbose	re.VERBOSE / re.X	Allows whitespace and comments in the pattern for readability
Unicode	re.UNICODE / re.U	Makes \w, \d, \s Unicode-aware (default in Python 3)

9. Escaping Special Characters

To match a metacharacter literally, escape it with a backslash: `\. \* \+ \? \( \) \[ \] \{ \} \| \^ \$ \\`

Example: to match a literal price like "$4.99", use: `\$\d+\.\d{2}`

10. Practical Examples

Purpose	Pattern	Notes
Email address	[\w.+-]+@[\w-]+\.[\w.-]+	Simplified — not full RFC 5322
Phone number	\+?\d[\d\s().-]{7,}\d	Loose match; allows spaces, dashes, parens
Simple username	^[a-zA-Z][a-zA-Z0-9_]{2,15}$	3-16 chars, starts with a letter
URL (basic)	https?://[\w.-]+\.[a-z]{2,}(/\S*)?	Matches http:// or https:// links
Date (YYYY-MM-DD)	\d{4}-\d{2}-\d{2}	No calendar validation, just format
Hex color code	#[0-9A-Fa-f]{6}\b	e.g. #1a2b3c
Whitespace trim check	^\s+|\s+$	Matches leading/trailing whitespace
Repeated word	\b(\w+)\s+\1\b	Uses a backreference, e.g. 'the the'

11. Quick Tips

• Regex is greedy by default — quantifiers grab as much as possible, then backtrack if needed. Add ? after a quantifier for lazy (minimal) matching.

• Character classes [ ] match exactly ONE character from the set — [abc] is NOT the same as (abc).

• Always escape special characters (. * + ? etc.) when you want them matched literally.

• Use non-capturing groups (?:...) when you need grouping but don't need to reference the match later — slightly better performance and cleaner captures.

• Test complex patterns against edge cases (empty strings, very long strings, unicode) before using them in production.

• For anything security- or validation-critical (like real email/phone validation), prefer a well-tested library over hand-rolled regex.